

Vision-Language Geometric Programming for Long-Horizon Robotic Assembly

Michael Shell, *Member, IEEE*, John Doe, *Fellow, OSA*, and Jane Doe, *Life Fellow, IEEE*

Abstract—The abstract goes here.

Index Terms—IEEE, IEEEtran, journal, LATEX, paper, template.

I. INTRODUCTION

THIS demo file is intended to serve as a “starter file” for IEEE journal papers produced under LATEX using IEEEtran.cls version 1.8b and later. I wish you the best of success.

mds

August 26, 2015

A. Subsection Heading Here

Subsection text here.

II. RELATED WORK

III. METHOD

We propose **V-LGP** (Vision-Language Geometric Programming), a hierarchical closed-loop framework that bridges high-level semantic reasoning with low-level geometric motion planning. The system operates through a “Perception-Reasoning-Action-Verification” pipeline, designed to solve long-horizon manipulation tasks. The workflow consists of three main phases:

A. System Overview

The architecture is designed to handle the complexity of multi-step assembly by decomposing the problem into semantic reasoning and geometric optimization.

1) *Initialization and Semantic Grounding (Phase 0–1)*: The system inputs include the *Current Scene Image* and a *Visual Assembly Manual*. The **VLM Matcher** first scans the workspace to semantically ground visual features (e.g. object colors, shapes) to simulation IDs. Simultaneously, the **VLM Analyst** parses the visual manual to construct a **Semantic Dependency Graph**. This graph decomposes the long-horizon mission into a topological sequence of sub-tasks (e.g., “Place Base” → “Insert Cylinder”).

M. Shell was with the Department of Electrical and Computer Engineering, Georgia Institute of Technology, Atlanta, GA, 30332 USA e-mail: (see <http://www.michaelshell.org/contact.html>).

J. Doe and J. Doe are with Anonymous University.

Manuscript received April 19, 2005; revised August 26, 2015.

2) *Receding Horizon Logic-Geometric Planning (Phase 2)*: To mitigate the combinatorial complexity of full-horizon planning, V-LGP employs a receding horizon strategy. The *Orchestrator* selects the next immediate sub-task from the dependency graph. The **VLM Strategist** then synthesizes the specific symbolic logic (encoded as ‘.lgp’ files and First-Order Logic) required for this step. This logic acts as a constraint for our underlying **LGP Solver** (see Sec. III-C), which optimizes a collision-free, geometrically feasible trajectory.

3) *Closed-Loop Execution and Verification (Phase 3–4)*: The optimized trajectory is executed by the low-level controller (Phase 3). Upon completion, the system captures an updated scene image. The **VLM Judicator** compares the current state against the visual manual. If the outcome is *Approved*, the dependency graph advances to the next node; if *Disapproved*, the **VLM Corrector** analyzes the failure mode and triggers a re-planning cycle. This visual feedback loop ensures robustness against execution errors.

B. VLM-Guided Task Decomposition

To address the combinatorial complexity of long-horizon assembly, we introduce a semantic decomposition layer that translates visual instructions into a structured Logic-Geometric problem.

1) *Semantic Precedence Graph Construction*: Unlike traditional TAMP methods that search for task sequences in a vast discrete space, V-LGP leverages the “common sense” physics knowledge of VLMs to infer assembly constraints directly from visual inputs. The VLM Analyst parses the manual into a **Layered Precedence Graph** $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, which organizes the long-horizon mission into a sequence of physically stable stages.

- **Nodes ($\mathcal{V}$)**: Each node $v_k \in \mathcal{V}$ represents an *Assembly Tier* (e.g., “Foundation”, “Second Layer”). A tier encapsulates a set of atomic actions $\mathcal{A}_k = \{a_{k,1}, a_{k,2}, \dots\}$ that can be potentially parallelized or solved within the same geometric phase.
- **Edges ($\mathcal{E}$)**: A directed edge $e_{i \rightarrow j}$ represents a *Physical Dependency*. In our JSON serialization (Fig. 1), this is explicitly encoded via the `placed_on` attribute. For instance, the transition from Node 1 to Node 2 signifies that the base in the second layer depends on the cylinders placed in the foundation tier for structural support.

This tiered structure allows the VLM to ground high-level spatial logic (e.g., `at_slot: top_left`) into a structured sequence that the LGP solver can consume incrementally.

As shown in the JSON output (Fig. 1), the VLM serializes this graph into a hierarchical list of **Assembly Nodes**. Each node contains specific action primitives and target slots. This structure effectively acts as a set of *Precedence Tiers* [2], where actions in Tier k are geometrically feasible only after Tier $k - 1$ is complete.

2) *Graph-to-Logic Translation*: The raw JSON output from the VLM is insufficient for the geometric solver. We implement a parser that converts the JSON description into two specific formats for the low-level solver:

- 1) **Logic State (.lgp)**: Defines the symbolic initial state and goal state for the current sub-task (e.g., `target: on(part_A, part_B)`).
- 2) **Geometric Constraints (.fol)**: Maps semantic descriptions (e.g., “top_left”) to precise frame IDs in the simulation (e.g., `frame_base_slot_01`).

This decomposition reduces a horizon- T problem into N smaller problems of horizon T/N , significantly improving the success rate of the underlying KOMO solver.

```
{
  "assembly_nodes": [
    {
      "node_id": 1, "description": "Foundation",
      "actions": [
        { "object": "base", "placed_on": "table",
          "at_slot": null },
        { "object": "cylinder", "placed_on": "base",
          "at_slot": "top_left" }
      ]
    },
    {
      "node_id": 2, "description": "Second Layer",
      "actions": [
        { "object": "base", "placed_on": ["cylinder"],
          "at_slot": null }
      ]
    }
  ]
  // ... (Tier n follows similar structure)
}
```

Fig. 1. The hierarchical JSON output from the VLM Analyst, representing the Tiered Assembly Graph. Each node groups actions that can be solved within a single LGP phase.

3) *Zero-Shot Prompting and Hallucination Mitigation*: To ensure the reliability of the VLM in a zero-shot setting, we implement a robust prompt engineering framework designed to minimize hallucinations and format non-compliance.

- **Cognitive Partitioning & Memory Flush**: We decompose the pipeline into specialized agents (Analyst, Strategist, Decider) to reduce context window saturation. Crucially, to prevent data leakage from few-shot examples, we enforce a *Memory Flush Protocol*, explicitly instructing the model to “wipe” visual memory of training examples before analyzing the live scene.
- **Deterministic Sandboxing**: We address the stochastic nature of LLM outputs via a hybrid parsing strategy. Critical code blocks (e.g., ‘.fol’ rules) are encapsulated in XML wrappers (e.g., `<FILE>`), allowing for robust extraction via regular expressions even if the surrounding JSON syntax is malformed.

- **Forensic Chain-of-Thought**: For the visual verification phase (Adjudicator), we enforce a “Forensic Report First” constraint. The model is required to generate a natural language analysis of spatial discrepancies before emitting the final structured verdict, forcing reasoning to precede decision-making.

C. Enhancements on Native LGP Solver

The native LGP framework, specifically the KOMO solver, provides an efficient mechanism for kinematic sequence verification in idealized simulations. However, it frequently overlooks the nuanced collision manifolds and precise grasping constraints required for robust physical interaction. Our work augments these geometric primitives, producing trajectories that are physically consistent and sufficiently smooth for direct deployment on real-world robotic platforms (i.e., bridging the Sim-to-Real gap). We implemented three classes of enhancements to robustify the solver.

1) *Virtual Anchor for Multi-Support Assembly*: Standard LGP defines placement as a relative pose constraint between two frames. This formulation fails when an object (e.g., a panel) must be placed on multiple independent supports (e.g., three cylinders) simultaneously. To resolve this, we introduced an *Intermediate Kinematic Formulation*. We dynamically calculate the centroid of all support surfaces and instantiate a temporary **Virtual Anchor** frame (A_v) with a free joint. The constraint is then reformulated as a two-stage chain: (1) The Virtual Anchor minimizes the pose error relative to the supports’ centroid, and (2) The payload object is kinematically switched to attach to A_v rather than any single support. This allows the solver to naturally distribute contact errors.

2) *Trajectory Shaping via Geometric Funnels*: Optimization-based planners often generate “snake-like” trajectories to minimize path length, causing objects to approach insertion points from lateral angles. This creates high risks of collision during Sim-to-Real transfer. We enforce **Pre-Action Funnels** for both picking and placing. For any insertion action at time t , we inject a strict waypoint constraint at $t - \delta$ (where $\delta = 0.1s$) that forces the object to align with the target’s Z -axis at a standoff distance (e.g., $+5cm$):

$$\Phi_{funnel}(x_t) = \sum_{i \in \{x, y\}} \|\text{pos}_i(O) - \text{pos}_i(Target)\|^2 + \|\text{vec}_z(O) \cdot \vec{z}_{world} - 1\|^2 \quad (1)$$

This effectively constructs a “funnel” that forces the optimizer to converge on vertical approach/retract paths, mirroring human assembly strategies.

3) *Phased Collision Avoidance and Lift-Off*: To prevent the “tunneling effect” where fast-moving objects pass through obstacles, we increased the solver’s temporal resolution from 10 to 30 steps per phase. Furthermore, we implemented **Context-Aware Constraints** to prevent “swiping” failures. Before initiating a lateral move, the solver performs an in-situ geometric check. If a previously placed object is detected within a 15cm radius, a *Lift-Off Constraint* is dynamically

activated, enforcing a pure vertical translation before any lateral motion is permitted. Additionally, collision constraints are relaxed hierarchically: strictly enforced during transit phases, but relaxed to allow contact (`OT_ineq` with margin 0) during the final grasp/place moments.

IV. EXPERIMENTAL RESULTS

1) Subsubsection Heading Here: Subsubsection text here.

V. CONCLUSION

The conclusion goes here.

APPENDIX A

PROOF OF THE FIRST ZONKLAR EQUATION

Appendix one text goes here.

APPENDIX B

Appendix two text goes here.

ACKNOWLEDGMENT

The authors would like to thank...

REFERENCES

- [1] H. Kopka and P. W. Daly, *A Guide to L^AT_EX*, 3rd ed. Harlow, England: Addison-Wesley, 1999.
- [2] Placeholder Author, "Title of the Fabrica Reference," Journal/Conference, Year.



Michael Shell Biography text here.

John Doe Biography text here.

Jane Doe Biography text here.